Practice · while loops

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What a loop is for

Why do we use a loop?

- To choose between two blocks of code.
- **Right answer:** To run the same block of code again and again, without copying it.
- To read a number from the keyboard.
- To make the program run faster.

Feedback: A loop repeats a block while a condition is true. Five `printf("Hello\n");` lines become one `printf` inside a loop. If you later need 1000 repeats, you change one number, not 1000 lines.

(mc) L1 · When the condition is checked

```
while (count <= 3) {
    printf("%d\n", count);
    count++;
}
```

When does C check the condition `count <= 3`?

- Only once, at the very start.
- After the last line of the program.
- **Right answer:** Before every run of the block, the first one too.
- Only when `count` changes.

Feedback: `while` checks first. True: run the block, then come back and check again. False: jump past the closing brace. So the condition is checked before every repeat, and one last time when it becomes false.

(mc) L1 · The four parts of a loop

```
int count = 1;
while (count <= 3) {
    printf("%d\n", count);
    count++;
}
```

Which line is the **update**, the line that moves the loop toward stopping?

- `int count = 1;`
- `while (count <= 3) {`
- `printf("%d\n", count);`
- **Right answer:** `count++;`

Feedback: Every loop has a start (`int count = 1;`), a check (`count <= 3`), a body (the `printf`) and an update (`count++;`). The update changes the variable, so one day the check becomes false and the loop ends.

(mc) L2 · The value after the loop

```
int x = 1;
while (x <= 4) {
    printf("%d ", x);
    x = x + 1;
}
```

The loop prints `1 2 3 4`. What is the value of `x` **after** the loop?

- 3
- 4
- **Right answer:** 5
- 1

Feedback: The loop only stops when `x <= 4` is false. That first happens when `x` is 5. So after a loop the variable holds the first value that failed the check, not the last value printed.

(mc) L2 · False at the start

```
int n = 10;
while (n < 5) {
    printf("Hello\n");
    n++;
}
printf("End\n");
```

What is printed?

- `Hello` five times, then `End`.
- `Hello` once, then `End`.
- **Right answer:** Only `End`.
- Nothing. The program never stops.

Feedback: `10 < 5` is false the very first time, so the block runs zero times. A `while` loop can run 0 times. The program goes straight to `End`.

(mc) L2 · The loop that never stops

```
int x = 1;
while (x <= 5) {
    printf("%d\n", x);
}
```

This compiles with no warning, even with `-Wall`. What happens when you run it?

- It prints `1` to `5` and stops.
- It prints `1` once and stops.
- **Right answer:** It prints `1` again and again and never stops.
- It prints nothing.

Feedback: Nothing in the block changes `x`, so `x` stays 1 and `1 <= 5` is true for ever. This is an infinite loop. The compiler does not warn you. Stop the program with **Ctrl+C** in the terminal, then add the missing `x++;`.

(fib) L3 · Type the keyword

Complete the keyword so the block repeats while the condition is true.

```
int count = 1;
[while] (count <= 3) {
    printf("%d\n", count);
    count++;
}
```

Feedback: `while`, then the condition in round brackets, then the block in braces. No semicolon after the round brackets.

(fib) L3 · Type the update

Complete the line so that the loop stops after printing `1 2 3 4 5`. Type it exactly, with the semicolon.

```
int x = 1;
while (x <= 5) {
    printf("%d ", x);
    [x++;|++x;|x+=1;|x += 1;|x=x+1;|x = x + 1;]
}
```

Feedback: Without an update the loop never ends. `x++;` adds 1 each time, so `x` reaches 6 and `6 <= 5` is false.

(fib) L3 · How many times

How many times does the block run? Type only the number.

```
int k = 10;
while (k > 0) {
    printf("%d ", k);
    k = k - 3;
}
```

The block runs [4] times.

Feedback: Trace it: `k` is 10, 7, 4, 1, and each of these is greater than 0, so the block runs 4 times and prints `10 7 4 1`. Then `k` becomes -2, `-2 > 0` is false, and the loop ends.